Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

High-Performance Computing Lab for CSE **2026**

Student: Riccardo Polo Discussed with: FULL NAME

Solution for Project 7

Due date: **Friday** 29 May 2026, 18:00 (Europe/Zurich)

HPC Lab for CSE 2025 — Submission Instructions

Following these instructions is mandatory: Submissions that do not comply **will not be considered**.

- **Submission Platform**: Assignments must be submitted via SAM-UP.
- **Required Files**: Your submission must include:
  - **Report**: A PDF summarizing your results and observations, based on this LaTeX template.
    - File name: `project_number_lastname_firstname.pdf`
    - You are allowed to discuss all questions with anyone you like; however: (i) your submission must list anyone you discussed problems with and (ii) you must write up your submission independently.
  - **Source Code Archive**: A **compressed tar archive**, named `project_number_lastname_firstname.tar.gz`, containing:
    - All source code files.
    - Build files and scripts (e.g., `Makefile`, batch job scripts, plot scripts, etc). If you modified any provided files, ensure they still work as expected.
    - The full LaTeX source of your report including all needed files to build the report (figures, listings, etc).
  - **Size Limit**: The total submission (PDF + archive) must not exceed **25 MB**.
  - **File Naming Rules**: Use only `ASCII` characters (avoid spaces, special symbols, and non-English letters).
- **Grading Process**: The TAs will evaluate your project based on:
  - Your report write-up.
  - Your code implementation.
  - Benchmarking your code's performance.

1. GPU architecture and performance characteristics [15 Points]

1.1. Peak performance [5 Points]

Listing 1: Output of the `deviceQuery` CUDA sample.

```
ripolo@eu-login-23:~/cuda-samples/Samples/1_Utilities/deviceQuery/build$
    srun --gpus=rtx_2080:1 ./deviceQuery
srun: job 66431453 queued and waiting for resources
srun: job 66431453 has been allocated resources
/cluster/home/ripolo/cuda-samples/Samples/1_Utilities/deviceQuery/build/./
    deviceQuery Starting...
```

```
 CUDA Device Query (Runtime API) version (CUDART static linking)

Detected 1 CUDA Capable device(s)

Device 0: "NVIDIA GeForce RTX 2080 Ti"
  CUDA Driver Version / Runtime Version          13.0 / 13.0
  CUDA Capability Major/Minor version number:    7.5
  Total amount of global memory:                 10821 MBytes (11346575360
      bytes)
  (068) Multiprocessors, (064) CUDA Cores/MP:    4352 CUDA Cores
  GPU Max Clock rate:                            1545 MHz (1.54 GHz)
  Memory Clock rate:                             7000 Mhz
  Memory Bus Width:                              352-bit
  L2 Cache Size:                                 5767168 bytes
  Maximum Texture Dimension Size (x,y,z)         1D=(131072), 2D=(131072,
     65536), 3D=(16384, 16384, 16384)
  Maximum Layered 1D Texture Size, (num) layers  1D=(32768), 2048 layers
  Maximum Layered 2D Texture Size, (num) layers  2D=(32768, 32768), 2048
     layers
  Total amount of constant memory:               65536 bytes
  Total amount of shared memory per block:       49152 bytes
  Total shared memory per multiprocessor:        65536 bytes
  Total number of registers available per block: 65536
  Warp size:                                     32
  Maximum number of threads per multiprocessor:  1024
  Maximum number of threads per block:           1024
  Max dimension size of a thread block (x,y,z): (1024, 1024, 64)
  Max dimension size of a grid size    (x,y,z): (2147483647, 65535, 65535)
  Maximum memory pitch:                          2147483647 bytes
  Texture alignment:                             512 bytes
  Concurrent copy and kernel execution:          Yes with 3 copy engine(s)
  Run time limit on kernels:                     No
  Integrated GPU sharing Host Memory:            No
  Support host page-locked memory mapping:       Yes
  Alignment requirement for Surfaces:            Yes
  Device has ECC support:                        Disabled
  Device supports Unified Addressing (UVA):      Yes
  Device supports Managed Memory:                Yes
  Device supports Compute Preemption:            Yes
  Supports Cooperative Kernel Launch:            Yes
  Device PCI Domain ID / Bus ID / location ID:   0 / 34 / 0
  Compute Mode:
     < Default (multiple host threads can use ::cudaSetDevice() with
        device simultaneously) >

deviceQuery, CUDA Driver = CUDART, CUDA Driver Version = 13.0, CUDA
   Runtime Version = 13.0, NumDevs = 1
Result = PASS
```

From listing 1 we can derive that:

- The number of Streaming Multiprocessors (SMs) is $n_{\text{SMs}} = 68$
- The number of CUDA cores per SM is $n_{\text{cores per SM}} = 64$
- The GPU maximumclock frequency is $f = 1.545$ GHz

From listing 1 we can also see that the CUDA Compute Capability is 7.5 and from reffff we get that

- For single precision (FP32) FMAs $TP_{\text{FP32}} = \frac{64}{64} = 1$
- For double precision (FP64) FMAs $TP_{\text{FP64}} = \frac{64}{2} = 32$

Thus we can caompute the theoretical peak performance of an NVIDIA GeForce RTX 2080 Ti for single and double precision as:

$$\begin{aligned} P_{\text{FP32}} &= n_{\text{SMs}} \cdot \frac{n_{\text{cores per SM}}}{TP_{\text{FP32}}} \cdot n_{FMA} \cdot f \\ &= 68 \cdot \frac{64}{1} \cdot 2\text{FP32} \cdot 1.545\text{GHz} = 13450,56 \text{ GFLOPS} \\ P_{\text{FP64}} &= n_{\text{SMs}} \cdot \frac{n_{\text{cores per SM}}}{TP_{\text{FP64}}} \cdot n_{FMA} \cdot f \\ &= 68 \cdot \frac{64}{32} \cdot 2\text{FP64} \cdot 1.545\text{GHz} = 420,33 \text{ GFLOPS} \end{aligned}$$

From Listing 1 we can get the following informations

- Global Memory : 10821 MiB = 10.57 GiB
- L2 Cache Size: 5.5 MiB
- L1 and Shared Memory per SM 96 KiB
- Read only memory 64 Kibytes
- Registers 64 Ki registers per block, each holding 4 bytes (total 256 KB).

We obtain the memory bus width $W_{\text{mem}} = 352$ bits and memory clock frequency as $f_{\text{mem}} = 7$ GHz. So we can compute the (theoretical) peak memory bandwidth of a GPU as:

$$\begin{aligned} B_{\text{GPU}} &= \frac{W_{\text{mem}}}{8} \cdot f_{\text{mem}} \cdot 2 \\ &= \frac{352}{8} \cdot 7\text{GHz} \cdot 2 = 616 \text{ GB/s} \end{aligned}$$

1.2. Performance model: A simple roofline model [5 Points]

Compute ridge points

$$I_{\text{ridge}} = \frac{P_{\text{peak}}}{B_{\text{GPU}}}$$

For single precision:

$$I_{\text{ridge, FP32}} = \frac{13450.56 \text{ GFLOPS}}{616 \text{ GB/s}} = 21.83 \text{ GLOP/byte}$$

For double precision:

$$I_{\text{ridge, FP64}} = \frac{420.33 \text{ GFLOPS}}{616 \text{ GB/s}} = 0.68 \text{ GLOP/byte}$$

We can retrieve a roofline plot for the NVIDIA GeForce RTX 2080 Ti remembering that $P_{\text{attainable}} = \min P_{\text{peak}}, B_{\text{GPU}} \cdot I_{\text{ridge}}$

2. Introduction to CUDA [50 Points]

2.1. Implement the simple kernels [30 Points]

For the double format we have following output:

Listing 2: Output of the simple kernels for the double format.

```
ripolo@eu-login-11:~/hpc_projects/project07/02cuda/simple_kernels$ srun --gpus=rtx_2080:1 ./benchmark
srun: job 171788 queued and waiting for resources
srun: job 171788 has been allocated resources
[benchmark] axpy: warmup=5 runs=25 avg_time=0.363086 ms
Compute utilization = 46.2073 GFLOPS/s
Memory utilization = 554.487 GB/s
first = 4, last = 4, expected = 4
[benchmark] add scaled diff: warmup=5 runs=25 avg_time=0.477494 ms
Compute utilization = 52.704 GFLOPS/s
```

```
Memory utilization = 562.176 GB/s
first = 1.6, last = 1.6, expected = 1.6
[benchmark] scaled diff: warmup=5 runs=25 avg_time=0.363736 ms
Compute utilization = 46.1247 GFLOPS/s
Memory utilization = 553.496 GB/s
first = 0.6, last = 0.6, expected = 0.6
[benchmark] scale: warmup=5 runs=25 avg_time=0.247412 ms
Compute utilization = 33.9054 GFLOPS/s
Memory utilization = 542.486 GB/s
first = 0.1, last = 0.1, expected = 0.1
[benchmark] linear combination: warmup=5 runs=25 avg_time=0.363924 ms
Compute utilization = 69.1512 GFLOPS/s
Memory utilization = 553.21 GB/s
first = 1.5, last = 1.5, expected = 1.5
ripolo@eu-login-11:~/hpc_projects/project07/02cuda/simple_kernels$
```

For float format we have the following output:

Listing 3: Output of the simple kernels for the float format.

```
ripolo@eu-login-11:~/hpc_projects/project07/02cuda/simple_kernels$ srun --
    gpus=rtx_2080:1 ./benchmark
srun: job 172188 queued and waiting for resources
srun: job 172188 has been allocated resources
[benchmark] axpy: warmup=5 runs=25 avg_time=0.182811 ms
Compute utilization = 91.7736 GFLOPS/s
Memory utilization = 550.642 GB/s
first = 4, last = 4, expected = 4
[benchmark] add scaled diff: warmup=5 runs=25 avg_time=0.239703 ms
Compute utilization = 104.988 GFLOPS/s
Memory utilization = 559.933 GB/s
first = 1.6, last = 1.6, expected = 1.6
[benchmark] scaled diff: warmup=5 runs=25 avg_time=0.182762 ms
Compute utilization = 91.798 GFLOPS/s
Memory utilization = 550.788 GB/s
first = 0.6, last = 0.6, expected = 0.6
[benchmark] scale: warmup=5 runs=25 avg_time=0.125165 ms
Compute utilization = 67.0205 GFLOPS/s
Memory utilization = 536.164 GB/s
first = 0.1, last = 0.1, expected = 0.1
[benchmark] linear combination: warmup=5 runs=25 avg_time=0.182696 ms
Compute utilization = 137.747 GFLOPS/s
Memory utilization = 550.989 GB/s
first = 1.5, last = 1.5, expected = 1.5
```

2.2. Implement the scalar product kernel [20 Points]

Listing 4: Output of the scalar product kernel for the double format.

```
ripolo@eu-login-20:~/hpc_projects/project07/02cuda/02_dot_product$ srun --
    gpus=rtx_2080:1 ./dot_prod
srun: job 191159 queued and waiting for resources
srun: job 191159 has been allocated resources
[benchmark] hpc_dot: warmup=5 runs=25 avg_time=27.3736 ms
Compute utilization = 1.22579 GFLOPS/s
Memory utilization = 9.80636 GB/s
Got: 16777216, expected: 16777216
```

3. Non-linear PDE mini-app with CUDA: The Fisher equation [35 Points]

output is

Listing 5: Output of the Fisher equation mini-app.

```
  ripolo@eu-login-20:~/hpc_projects/project07/03_fisher$ srun --gpus=
     rtx_2080:1 ./main 128 100 0.005
srun: job 192180 queued and waiting for resources
srun: job 192180 has been allocated resources
================================================================================

                       Welcome to mini-stencil!
version   :: CUDA
mesh      :: 128 * 128 dx = 0.00787402
time      :: 100 time steps from 0 .. 0.005
iteration :: CG 300, Newton 50, tolerance 1e-06
================================================================================

--------------------------------------------------------------------------------

simulation took 0.221755 seconds
1516 conjugate gradient iterations, at rate of 6836.37 iters/second
300 Newton iterations
--------------------------------------------------------------------------------

### 128, 100, 1516, 300, 0.221755 ###
Goodbye!
```

Using float the output cd